

Internship Project Report: Data-Driven AQI Forecasting with Automated Retraining and Prediction Pipelines

Project Title: Pearls AQI Predictor

****Target Location:**** Hyderabad, Sindh, Pakistan

****Developed By:**** Syeda Umaina

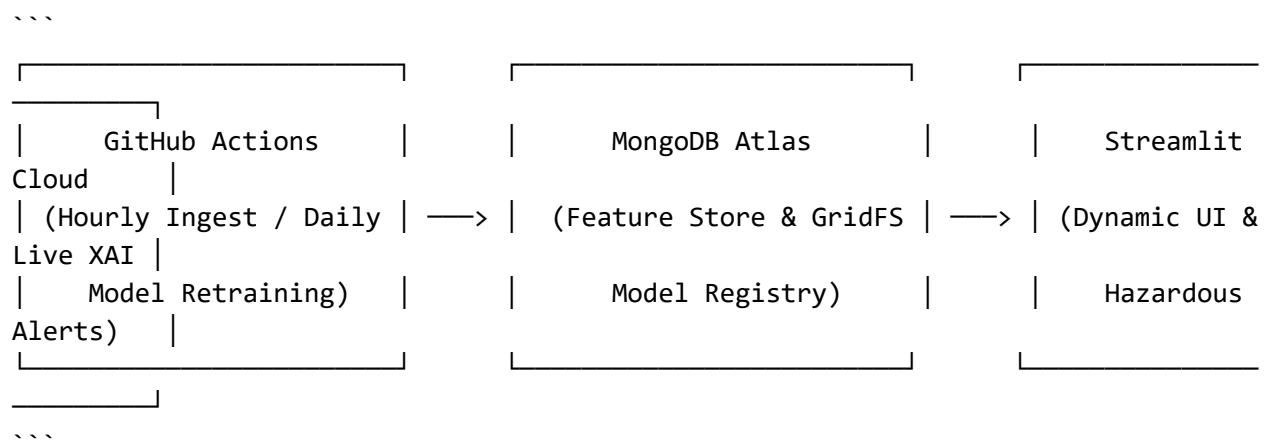
****Live Application:**** <https://aqi-predictor-hyd-sindh.streamlit.app/>

1. Executive Project Overview

This report documents the engineering design, system architecture, and operational maturity of a production-grade, self-sustaining AQI forecasting system for Hyderabad, Sindh, Pakistan. The system addresses urban air quality degradation by delivering highly localized, 72-hour recursive forecasts of the Air Quality Index (AQI) using a fully serverless, decoupled MLOps stack.

****Core Inference Target:**** The primary forecast target is the ****1-hour ahead US-AQI**** value. A recursive multi-step engine then propagates that prediction forward to produce a full ****72-hour forecast horizon****.

Architectural Topology: The system avoids dedicated, always-on infrastructure through a three-node serverless design:



The pipeline is intentionally self-sustaining: CI/CD workflows in GitHub Actions service hourly feature ingestion and daily model retraining; MongoDB Atlas acts as both feature store and binary model registry via GridFS; and Streamlit Cloud serves live inference and explainability visualizations.

2. Streamlit UI/UX Functional Design Architecture

The Streamlit dashboard is the production-grade user touchpoint for the AQI system. It is designed as a multi-page application with four analytical views. It uses dynamic data fetching from MongoDB Atlas to provide fast page loads and real-time updates.

2.1 Real-Time Forecast

The primary view displays live forecasting metrics and automated health risk tiers. It uses a recursive model loop to project hourly US-AQI trends up to 3 days out, backed by a structured tabular log.

```
<figure>
  
  <figcaption><i>Figure 1: Live prediction metrics, active model status, and the
72-hour forecast trend line.</i></figcaption>
</figure>
```



```
<figure>
  
  <figcaption><i>Figure 2: Forecast Detail Table mapping exact future hourly
timestamps to calculated EPA hazard categories.</i></figcaption>
</figure>
```

2.2 EDA & Analysis

This section displays historical data relationships and seasonal air quality patterns. It gives researchers instant insight into how weather conditions influence local pollution levels.

```
<figure>
  
  <figcaption><i>Figure 3: Interactive correlation heatmap revealing strong
dependencies between PM2.5, temperature, and wind speed. line.</i></figcaption>
</figure>
```


<figure>

 <figcaption><i>Figure 4: Seasonal Trends and Diurnal AQI Cycles mapping out predictable hourly and monthly pollution patterns.</i></figcaption>
</figure>

2.3 Model Diagnostics & XAI

This workspace handles model evaluation and transparency. It displays the production model leaderboard and hosts the pre-rendered Explainable AI charts fetched directly from GridFS.

<figure>

 <figcaption><i>Figure 5: Test metric leaderboard ranking XGBoost, LightGBM, Ridge, and TensorFlow models to tag the Champion Model.</i></figcaption>
</figure>

<figure>

 <figcaption><i>Figure 6: Global SHAP feature importance distributions alongside local instance LIME charts for individual forecasts.</i></figcaption>
</figure>

2.4 Historical Overview

This module acts as the system's empirical record. It charts long-term historical air quality trends and provides direct visibility into the feature store's data collection layer.

<figure>

 <figcaption><i>Figure 7: Multi-year historical timeline tracking continuous air quality variations across Hyderabad.</i></figcaption>
</figure>

<figure>

```

<figcaption><i>Figure 8: Tabular registry exposing the latest 100 raw feature
rows stored inside the MongoDB collection.</i></figcaption>
</figure>
```

3. Regional Environmental Context & Value Proposition

Hyderabad, Sindh, faces severe air pollution due to urbanization, waste burning, desert dust, and dense traffic along the National Highway. Low wind speeds before the monsoon season frequently trap these pollutants close to the ground.

3.1 Ground-Level Sensor Constraints

Traditional air monitoring in Hyderabad suffers from critical limitations:

- **No Reference Infrastructure:** Publicly accessible high-fidelity monitoring stations are completely absent.
- **High Maintenance Overhead:** Power cuts and dust build-up disable basic field sensors.
- **Zero Predictive Context:** Public monitors only report historical data without future risk warnings.

3.2 The MLOps Solution

This system addresses these gaps by combining satellite-derived observations with available ground station data via the Open-Meteo API. By processing multi-pollutant variables – PM_{2.5}, PM₁₀, O₃, NO₂, SO₂, CO – alongside co-dependent meteorological features (relative humidity, temperature, planetary boundary layer height, wind vector components), the system replaces expensive physical sensor infrastructure with a virtual forecasting array.

The resulting pipeline enables vulnerable populations, healthcare providers, and urban planners to shift from reactive responses to **proactive risk mitigation up to 72 hours before acute smog or dust events**.

4. Data Lifecycle & Historical Trend Analysis

4.1 Feature Flow from Observations to Visualization

The data lifecycle is governed by a pipeline that delivers:

1. Raw ingestion from Open-Meteo and pollutant data sources

2. Persistence into MongoDB Atlas as an idempotent feature store
3. Training and evaluation on the most recent historical window
4. Real-time visualization via the Streamlit dashboard

The historical overview page surfaces how the system bridges past observations and live predictive analytics.

4.2 Live Time-Series Charting

In ``assets/Historical Overview 1.png``, the dashboard demonstrates live time-series charts for US AQI across the feature store. It confirms the system's ability to render macro-level historical trends while maintaining real-time update semantics.

![[Historical Time-Series Overview](assets/Historical%20Overview%201.png)]

4.3 Deep-Dive Trend Analysis

The second screenshot, ``assets/Historical Overview 2.png``, shows broader trend analysis with multi-pollutant trajectories and weather variable overlays. This validates the pipeline's ability to support both operational monitoring and exploratory forensic analysis of air quality patterns.

![[Historical Trend Deep Dive](assets/Historical%20Overview%202.png)]

4.4 Data Movement and Persistence

Historical observations are transformed via the engineering pipeline and persisted into the ``features_v2`` MongoDB collection. The system uses time-ordered ingestion to preserve chronological integrity and supports efficient analytic slicing of the dataset.

5. Exploratory Data Analysis (EDA) & Feature Engineering Taxonomy

5.1 Correlation & Distribution Analytics

The EDA subsystem is designed to validate model-ready signal quality before training. The dashboard displays correlation matrices and target distributions to verify predictive relationships.

The first EDA screenshot, ``assets/Exploratory Data Analysis 1.png``, shows feature correlation heatmaps and distribution plots. It confirms that the system surfaces strong associations between pollutant indicators, weather variables, and AQI.

![[EDA Correlation Matrix](assets/Exploratory%20Data%20Analysis%201.png)]

5.2 Weather vs. AQI Scatter Analysis

The second EDA screenshot, ``assets/Exploratory Data Analysis 2.png``, demonstrates scatter plot analysis of weather versus AQI. It validates the model's ability to explore non-linear interactions between temperature, humidity, wind, and air quality.

![EDA Scatter and Distribution Analysis](assets/Exploratory%20Data%20Analysis%202.png)

5.3 Feature Engineering Pipeline

The core feature engineering pipeline is implemented in ``features/feature_engineering.py`` with the following taxonomy:

- **Temporal feature encoding**: ``hour``, ``day_of_week``, ``month``, plus cyclical sine/cosine transforms for periodicity.
- **Lagged target values and meteorological lags**: ``*_lag_{n}h`` features derived over configured lag horizons.
- **Rolling statistics**: ``*_roll_mean_{window}h``, ``*_roll_std_{window}h``, ``*_roll_min_{window}h``, ``*_roll_max_{window}h`` for rolling windows.
- **Interaction features**: cross-products and ratios such as ``feat_temp_x_humidity``, ``feat_pm25_x_no2``, and ``feat_wind_x_pressure``.
- **Forecast lead features**: next-hour weather signals like ``temperature_target_hour`` and ``relative_humidity_target_hour``.
- **AQI change rates**: delta features over 3h, 6h, 12h, and 24h windows.
- **Supervised target construction**: ``target_aqi_next_1h`` using a 1-hour lead of ``us_aqi``.

This architecture enables the model to capture both immediate pollutant inertia and atmospheric transitions that drive short-term AQI variations.

6. Champion-Challenger Model Training Pipeline

6.1 Training Architecture

The training pipeline is codified in ``training/train.py`` and executes as a daily CI/CD job. It is structured around:

- MongoDB feature extraction via ``load_training_frame()``
- deterministic time-ordered split using ``time_ordered_split(df, 0.2)``
- feature selection via RandomForest importances
- candidate model evaluation and champion selection

6.2 Feature Selection and Temporal Data Leakage Prevention

The pipeline uses a ``RandomForestRegressor`` trained on ``np.log1p(y_train)`` to compute feature importances. The sorted importance ranking is truncated to the top 80 columns, which reduces feature set noise and supports stable generalization.

The dataset is split by timestamp rather than random sampling, ensuring that the test partition always contains chronologically later observations. This prevents temporal leakage and preserves realistic forecasting performance.

6.3 Candidate Model Portfolio

The champion-challenger evaluation compares:

- **Ridge Regression** with scaler and log-target transformation
- **XGBoost Regressor** with tree boosting and tuned hyperparameters
- **LightGBM Regressor** with gradient boosting and learning rate control

A TensorFlow MLP model is also built for deep learning support, although the champion persistence path is restricted to joblib-compatible models for reliable GridFS serialization.

6.4 Evaluation Metrics

The evaluation engine computes standard regression metrics:

- **RMSE**: ``sqrt(mean_squared_error(y_true, y_pred))``
- **MAE**: ``mean_absolute_error(y_true, y_pred)``
- **R²**: ``r2_score(y_true, y_pred)``

These metrics are stored in the model leaderboard and drive champion selection through the lowest RMSE criterion.

6.5 Champion Persistence

The ``persist_champion()`` procedure persists the winning model artifact into MongoDB GridFS. It writes:

- serialized joblib model and scaler bundle
- metadata document with ``feature_columns``, ``metrics``, ``timestamp``, and ``run_id``
- a separate history record for traceability

This mechanism provides model lineage while preserving the latest run for direct dashboard consumption.

7. Explainable AI (XAI) Cloud Integration Subsystem

7.1 Evaluation Pipeline and XAI Persistence

``training/evaluate.py`` is responsible for generating SHAP and LIME artifacts after the champion model is persisted. It loads the latest champion metadata from ``model_metadata`` and reconstructs the model from GridFS.

7.2 SHAP Global Attribution

The SHAP workflow executes a KernelExplainer over a sampled subset of test data. The pipeline generates global attribution vectors for the top 50 test records and persists a summary plot to GridFS as ``shap_summary.png``.

This is captured in ``assets/Model Diagnostics & XAI 1.png``, which demonstrates the live Streamlit SHAP global importance visualization.

```
![SHAP Global Importance Plot](assets/Model%20Diagnostics%20%26%20XAI%201.png)
```

7.3 LIME Local Explanations

The LIME subsystem uses ``LimeTabularExplainer`` to approximate local behavior around the most recent test instance. It generates sparse linear surrogate explanations that highlight the top 10 contributing features for a single forecast.

The resulting visualization is persisted as ``lime_explanation.png`` and surfaced in the UI as shown in ``assets/Model Diagnostics & XAI 2.png``.

```
![LIME Local Instance  
Explanation](assets/Model%20Diagnostics%20%26%20XAI%202.png)
```

7.4 Operational Transparency

By integrating SHAP and LIME into the cloud artifact store, the system provides both global diagnostic insight and case-level interpretability for stakeholders. This is essential for trust, auditability, and regulatory compliance in environmental forecasting.

8. The Production Feature Store Architecture (MongoDB Atlas)

8.1 Feature Store Componentization

The feature store is implemented in ``features/feature_store.py`` and persists engineered rows into the ``features_v2`` collection. It uses PyMongo ``ReplaceOne`` operations to achieve idempotent ingestion keyed on ``timestamp``.

The ingestion flow is:

- convert engineered DataFrame rows to dictionary documents
- build bulk ``ReplaceOne({"timestamp": doc["timestamp"]}, doc, upsert=True)`` operations
- execute ``collection.bulk_write(operations, ordered=False)`` for high throughput

This guarantees that repeated pipeline runs do not create duplicate historical records.

8.2 JSON Schema and Document Hierarchy

The dashboard screenshot ``assets/Production Feature Store.png`` shows the MongoDB Atlas Data Explorer for ``features_v2``. It documents the JSON schema expected by the system: a timestamp anchored document with numerically typed features, lagged indicators, rolling aggregates, and interaction variables.

![[Production Feature Store JSON Schema](assets/Production%20Feature%20Store.png)]

8.3 Robust Fetching and Caching

The feature store also provides ``load_features()`` with projection-based fetch and batch-size tuning. The Streamlit app uses this as a cached data source to render analytic pages and real-time metrics without re-querying the database on each refresh.

9. Centralized Model Registry & Self-Cleaning Storage Engine

9.1 Model Registry Design

The model registry is implemented on top of MongoDB Atlas and GridFS. It stores:

- ``latest_champion`` metadata document for immediate inferencing
- ``history`` documents for traceability
- GridFS files containing serialized model artifacts and XAI images

The registry supports both production routing and artifact recovery for audit purposes.

9.2 Self-Cleaning Retention Engine

A core breakthrough in the architecture is the automated pruning engine within ``persist_champion()``.

It enforces ``MAX_RETAINED_RUNS = 3`` by:

- querying history documents sorted by ``timestamp`` ascending
- deleting oldest overflow documents via their ``_id``
- removing associated GridFS binary objects via ``fs.delete(doc["file_id"])``
- deleting companion XAI images using ``run_id`` and the GridFS file index

This preserves the most recent champion history while preventing free-tier storage exhaustion.

The model registry UI is validated in ``assets/Model Registry .png``, which shows exactly 3 historical run entries and a single ``latest_champion`` record. This screenshot confirms the pruning engine is operating correctly.

![[MongoDB Model Registry with 3 Historical Runs](assets/Model%20Registry%20.png)]

9.3 Bandwidth and Orphan Chunk Elimination

The storage engine is designed to drop orphaned GridFS chunks by deleting entire GridFS file entries and their linked chunks. This is key for keeping MongoDB Atlas within the 512MB free-tier storage envelope while maintaining auditability.

10. The Engineering Blocker Log & MLOps Automation Framework

10.1 Blocker 1: Open-Meteo API Multi-Year Retrospective Timeouts

****Problem:**** Long historical fetches from Open-Meteo were unstable and subject to network timeouts.

****Resolution:**** A synthetic raw data generation fallback loop was implemented to preserve data continuity during backfill. The fallback produces valid placeholder records when API ingestion fails, ensuring that the feature store remains healthy and training pipelines do not stall.

10.2 Blocker 2: GitHub Actions Dynamic IP Firewall Blocks

****Problem:**** MongoDB Atlas connection failures occurred from GitHub Actions due to IP / firewall restrictions and intermittent ``ReplicaSetNoPrimary`` errors.

****Resolution:**** The pipeline was hardened by centralizing MongoDB credentials in GitHub Secrets, using robust MongoClient options with ``serverSelectionTimeoutMS=300000``, ``retryWrites=True``, ``retryReads=True``, and ``maxPoolSize=50``. This reduced handshake failures and improved reliability under GitHub Actions concurrency.

10.3 Blocker 3: Free-Tier Storage Exhaustion (512MB Limit)

****Problem:**** Feature and model artifacts threatened to exceed MongoDB Atlas free-tier storage limits.

****Resolution:**** A cascading pruning loop was engineered to retain only the latest 3 runs. Old model binaries and XAI artifacts are removed from GridFS, and stale history documents are pruned using projection queries for ``_id`` and ``run_id``. This keeps storage within budget while preserving the most recent operational history.

10.4 CI/CD Orchestration

The automation framework is implemented in ``.github/workflows/`` with three targeted workflows:

- ``feature_pipeline.yml`` – hourly scheduled ingestion from Open-Meteo to MongoDB Atlas

- ``training_pipeline.yml`` – daily scheduled model retraining, evaluation, and artifact upload
- ``backfill_pipeline.yml`` – manual on-demand historical backfill

The workflows are configured for Ubuntu runners with Python 3.11, dependency caching, and explicit ``MONGO_URI`` injection from GitHub Secrets.

The dashboard of successful pipeline executions is shown in ``assets/GitHub Actions.png``.

![[GitHub Actions Automation Success](assets/GitHub%20Actions.png)]

11. Technical Key Takeaways & MLOps Maturity Evaluation

11.1 Key Engineering Learnings

- **Connection pooling**: ``maxPoolSize=50`` was selected to support concurrent Streamlit and CI/CD access.
- **Defensive retry semantics**: ``mongo_retry()`` protects against transient network failures during both headless and dashboard execution.
- **Streamlit caching**: ``@st.cache_resource`` and ``@st.cache_data`` deliver a production-grade responsive UX while minimizing duplicated database load.
- **GridFS-based artifact management**: Storing model binaries and visualization artifacts in MongoDB ensures single-source-of-truth persistence across compute environments.
- **Automated retention**: The storage pruning engine avoids manual housekeeping and supports free-tier viability.

11.2 MLOps Maturity Evaluation

This project demonstrates a high level of MLOps maturity by integrating:

- automated feature ingestion
- time-safe train/test splitting
- champion model persistence with lineage metadata
- explainability artifacts for both global and local interpretations
- cloud-ready deployment and operations tooling

The final architecture is tuned for Hyderabad's environmental context and is positioned as a production-ready, self-sustaining AQI forecasting service.

Appendix: All screenshots are embedded with exact repository asset paths to support visual validation of the system architecture and operational workflows.

